MACHINE LEARNING

LESSON 04:

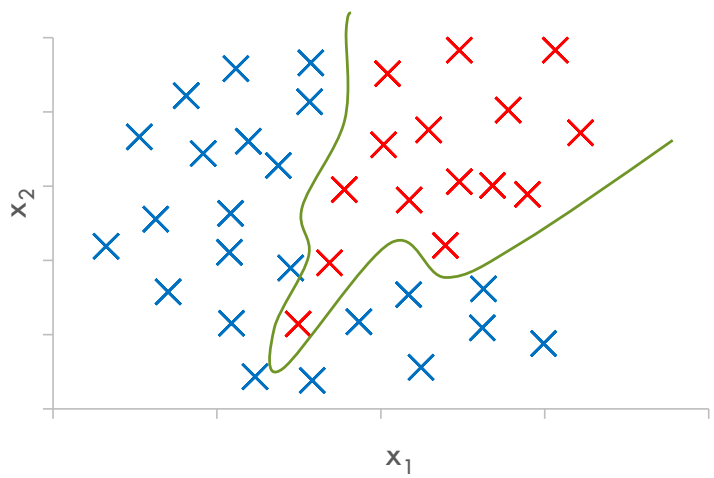
Neural Networks

Prepared by Dr. Dona Valy

Inspired by and adapted from the Coursera Machine Learning course by Andrew Ng (coursera.org/learn/machine-learning)

NON LINEAR HYPOTHESES

- $h(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2 + \theta_4 x_1^2 x_2 + \theta_5 x_1 x_2^2 + \cdots)$



NON LINEAR HYPOTHESES

- Image classification

8x8 pixel image

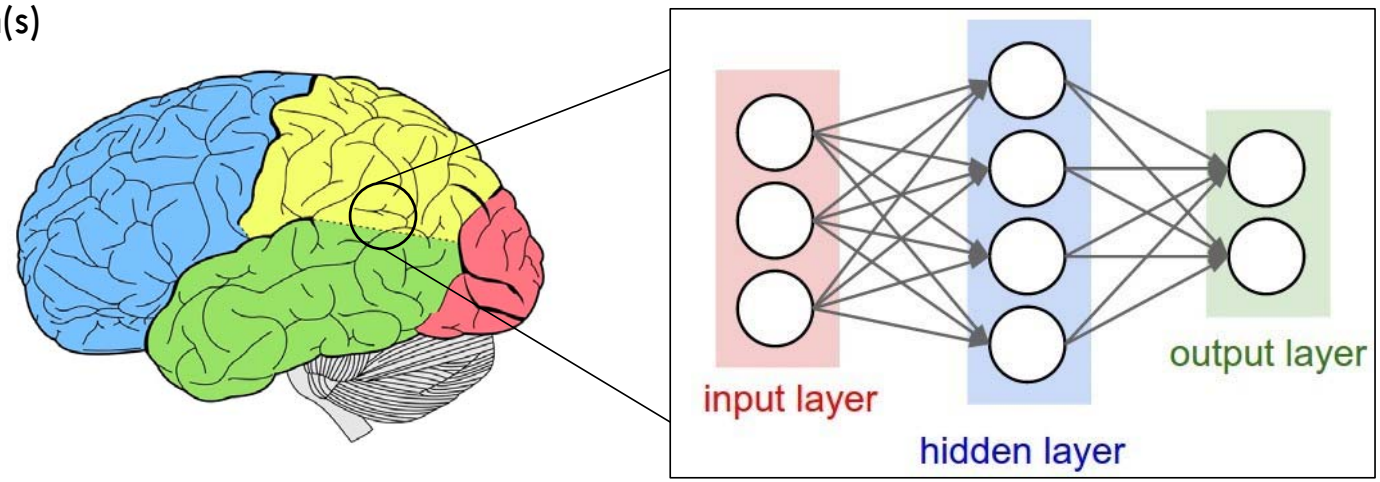
0	0	1	1	1	1	0	0
0	1	0	0	0	0	1	0
1	0	1	0	1	0	0	1
1	0	1	0	1	0	0	1
1	0	0	0	0	1	0	1
1	0	1	1	1	0	0	1
0	1	0	0	0	0	1	0
0	0	1	1	1	1	0	0

- 8x8 pixel image ⇒ 64 pixel values as features
- Number of features $n = 64$

$$x = \begin{bmatrix} pixel\ 1\ value \\ pixel\ 2\ value \\ \vdots \\ pixel\ n\ value \end{bmatrix}$$

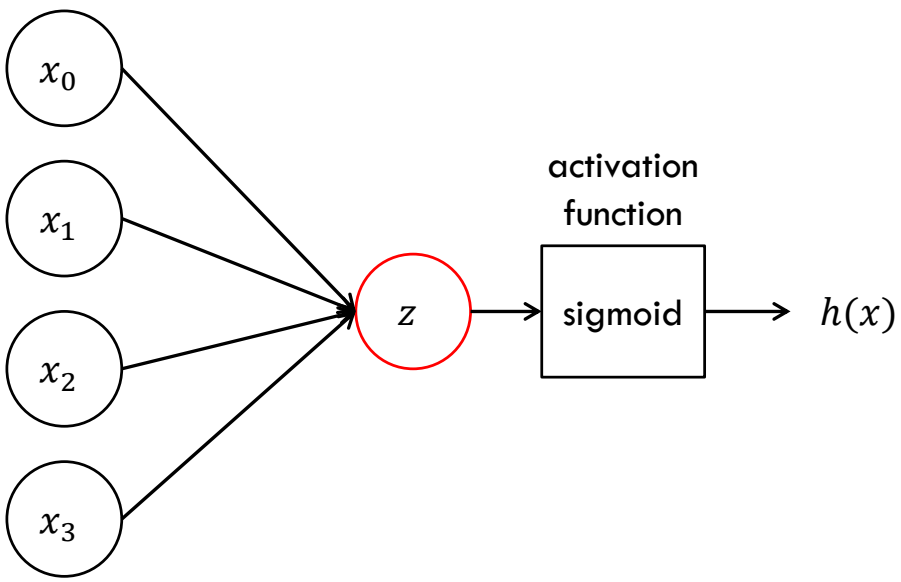
ARTIFICIAL NEURAL NETWORKS (ANN)

- **Idea:** to make the machine more robust, intelligent, and able to learn, let's model it after ***human brains***
- Biological Inspiration: the brain is composed of neurons
  - A neuron receives input from other neurons
  - Inputs are approximately summed
  - When the input exceeds a threshold the neuron sends an electrical spike that travels to the next neuron(s)



NEURON MODEL

- Logistic unit



$$z = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$
$$h(x) = a = g(z)$$

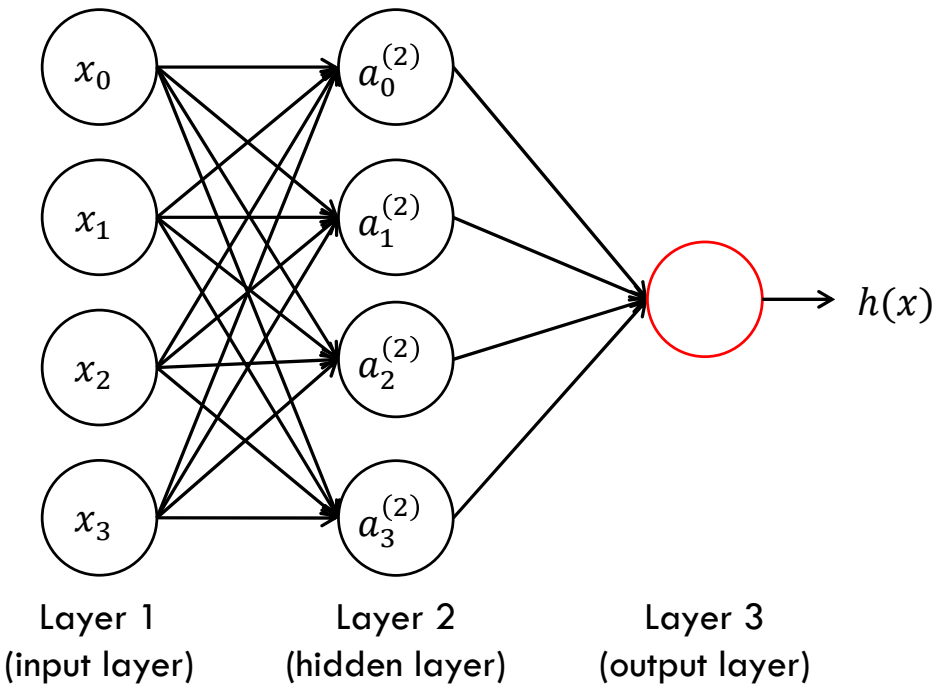
$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}, \theta = [\theta_0 \;\; \theta_1 \;\; \theta_2 \;\; \theta_3]$$

$$z = \theta x$$

1x4 4x1

NEURAL NETWORK MODEL

- Neural network layers



Notation:

- $a_i^{(j)}$ = activation of unit i in layer j
- $\Theta^{(j)}$ = weight matrix mapping from layer j to layer $j+1$

apply element-wise

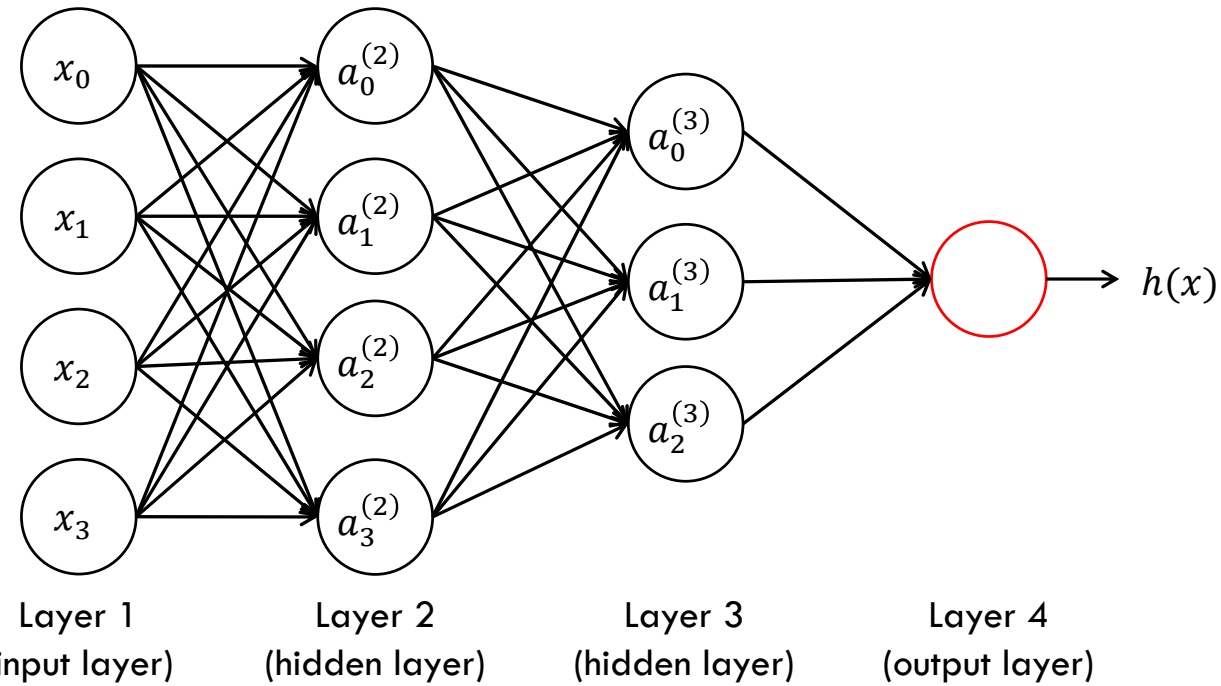
$$a^{(2)} = g(\Theta^{(1)} x)$$

4x1 4x4 4x1

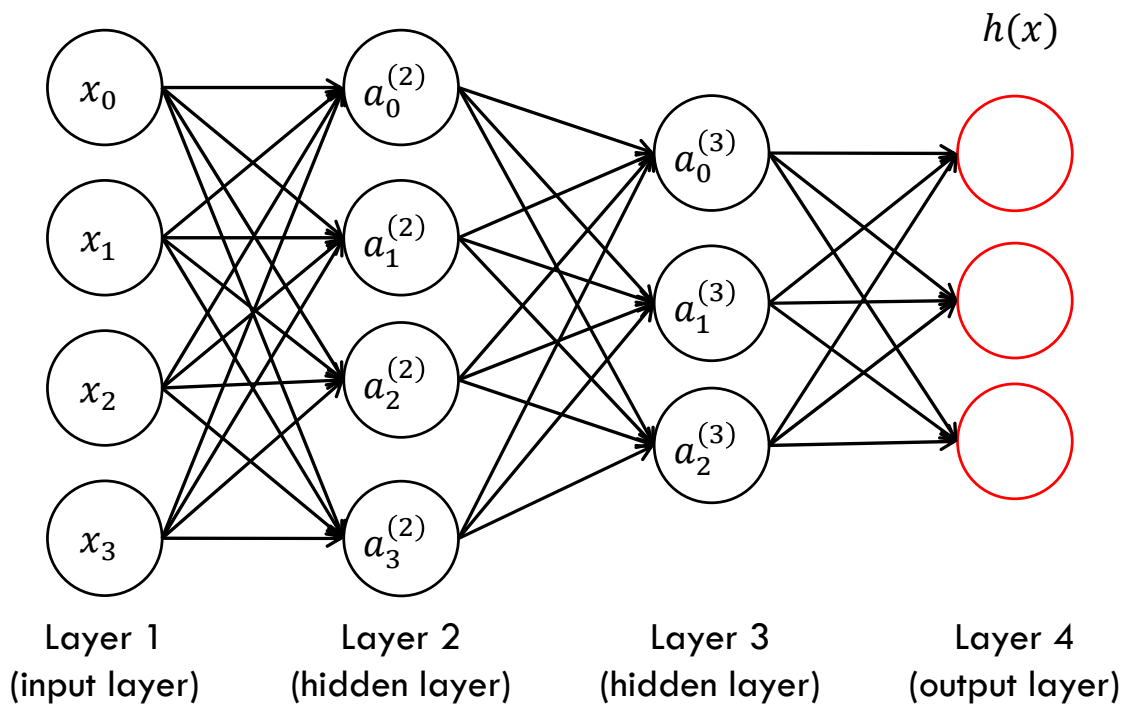
$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}, \Theta^{(1)} = \begin{bmatrix} \theta_{00}^{(1)} & \theta_{01}^{(1)} & \theta_{02}^{(1)} & \theta_{03}^{(1)} \\ \theta_{10}^{(1)} & \theta_{11}^{(1)} & \theta_{12}^{(1)} & \theta_{13}^{(1)} \\ \theta_{20}^{(1)} & \theta_{21}^{(1)} & \theta_{22}^{(1)} & \theta_{23}^{(1)} \\ \theta_{30}^{(1)} & \theta_{31}^{(1)} & \theta_{32}^{(1)} & \theta_{33}^{(1)} \end{bmatrix}$$

NEURAL NETWORK MODEL

- Neural network layers



NEURAL NETWORK FOR MULTICLASS CLASSIFICATION



y becomes ***one hot encoding***

It is either

$$\begin{bmatrix}1\\0\\0\end{bmatrix}, \begin{bmatrix}0\\1\\0\end{bmatrix}, \begin{bmatrix}0\\0\\1\end{bmatrix}$$

NEURAL NETWORK FOR MULTICLASS CLASSIFICATION

$$y^{(i)} = \begin{bmatrix} y_1^{(i)} \\ y_2^{(i)} \\ \vdots \\ y_K^{(i)} \end{bmatrix}, h\left(x^{(i)}\right) = \begin{bmatrix} h\left(x^{(i)}\right)_1 \\ h\left(x^{(i)}\right)_2 \\ \vdots \\ h\left(x^{(i)}\right)_K \end{bmatrix}$$

where $y_k^{(i)} = \{0,1\}$, K the number of classes

- Cost Function
  - $J(\Theta) = \frac{1}{m}\left[\sum_1^m \sum_1^K -y_k^{(i)} \log\left(h\left(x^{(i)}\right)_k\right) - \left(1 - y_k^{(i)}\right) \log\left(1 - h\left(x^{(i)}\right)_k\right)\right]$

- Note: the value of $h\left(x^{(i)}\right)_k$ should be $0 \leq h\left(x^{(i)}\right)_k \leq 1$

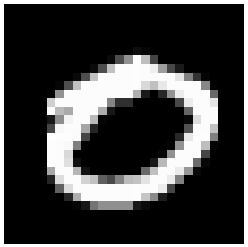
EXERCISE 4.1

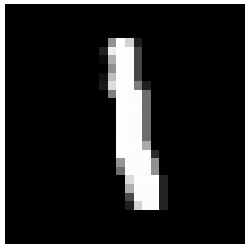
Create a neural network model with 2 hidden layers (each layer consists of 5 units) as a solution of:

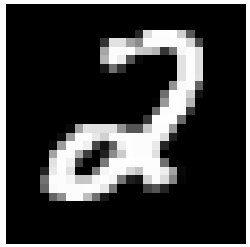
1. Exercise 3.1.1
2. Exercise 3.2

EXERCISE 4.2

Build a neural network model to classify images of handwritten digits. The dataset can be found in 'tiny_mnist.zip'. There are 3 classes: 0, 1, and 2.

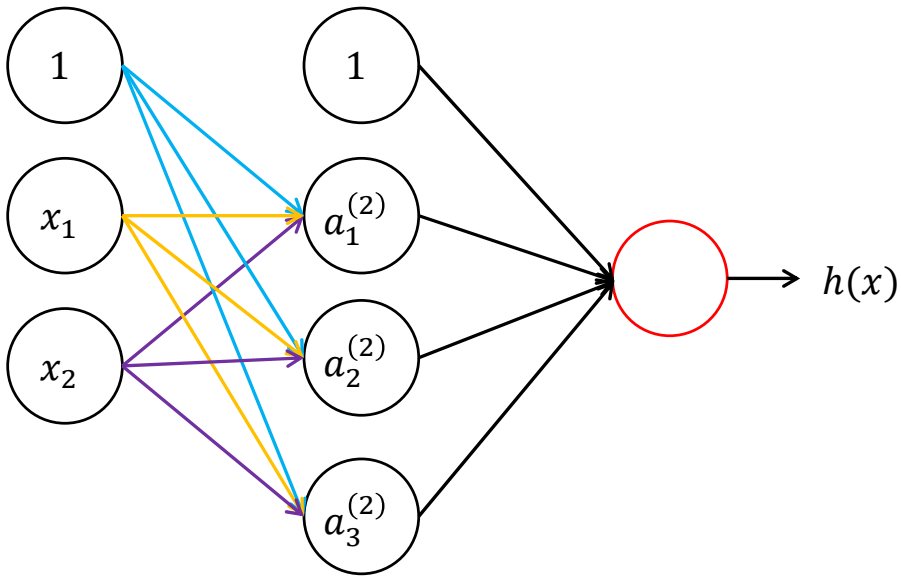






PARAMETER INITIALIZATION

- Zero initialization



After each update, parameters corresponding to inputs going into each of the hidden units are identical

$$\Rightarrow a_1^{(2)} = a_2^{(2)} = a_3^{(2)}$$

SOLUTION: Random Initialization (Symmetry Breaking)
$\Rightarrow$ Initialize each Θ_{ij} to a random value in $[-\epsilon, \epsilon]$ where ϵ is a small value

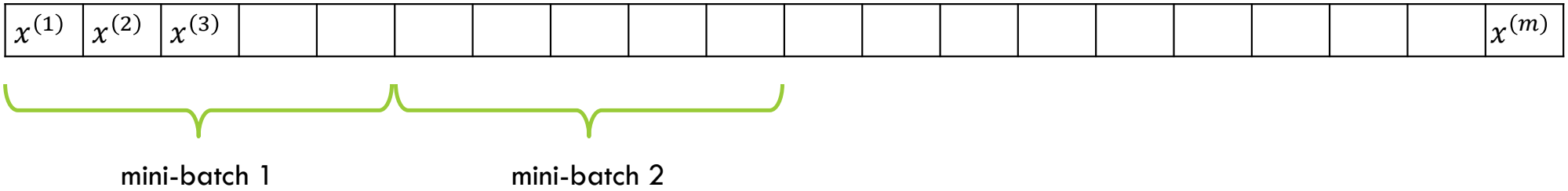
SUMMARY: BUILDING A NEURAL NETWORK

- Pick a network architecture
- Randomly initialize parameters
- Implement forward propagation to get $h(x)$
- Implement cost function $J(\Theta)$
- Minimize $J(\Theta)$

- TRAINING LOOP: for each iteration
  - Perform forward propagation
  - Compute cost using $J(\Theta)$
  - Use gradient descent by implementing backpropagation to compute partial derivatives $\frac{\partial}{\partial \Theta_{ij}} J(\Theta)$
  - Update parameters

MINI-BATCH GRADIENT DESCENT

- 3 variants of gradient descent
  - **Batch GD:** update parameters after going through all training examples (1 epoch)
  - **Stochastic GD:** update parameters after each training example
  - **Mini-batch GD:** update parameters after a small batch of training examples (1 epoch might contain multiple mini-batches)



EXERCISE 4.3

Given the complete MNIST dataset (can be found in "sample_data" of Google Colab), build another neural network. There are now 10 classes.

Hint: try to train the network using a small batch from the whole training set for each iteration.